

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 3**



**Single Link List
Oleh:**

Nama Siti Nurchalisah

NIM.2510817320002

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 3

Laporan Praktikum Algoritma & Struktur Data Modul 3 Single link list ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Siti Nurchalisah
NIM : 2510817320002

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Fadhil Syahdama Mahatma Putra
NIM. 2410817210026

Andreyan Rizky Baskara, S.Kom., M.Kom
NIP. 199307032019031011

DAFTAR ISI

LEMBAR PENGESAHAN.....	II
DAFTAR ISI	III
DAFTAR GAMBAR	V
DAFTAR TABEL	VII
SOAL 1	1
A. Source Code.....	2
B. Output Program.....	9
C. Pembahasan.....	9
SOAL 2	11
A. Output Program.....	11
B. Pembahasan.....	11
SOAL 3	12
A. Output Program.....	12
B. Pembahasan.....	12
SOAL 4	13
A. Output Program.....	13
B. Pembahasan.....	13
SOAL 5	14
A. Output Program.....	14
B. Pembahasan.....	14
SOAL 6	15
A. Output Program.....	15
B. Pembahasan.....	15
SOAL 7	16
A. Output Program.....	16
B. Pembahasan.....	16

SOAL 8	17
A. Output Program.....	17
B. Pembahasan.....	17
SOAL 9	18
A. Output Program.....	18
B. Pembahasan.....	18
SOAL 10	19
A. Output Program.....	19
B. Pembahasan.....	19
LINK GITHUB.....	20

DAFTAR GAMBAR

Gambar 1. Screenshot Output Program Soal 1.....	9
Gambar 2. Screenshot Output Program Soal 2.....	11
Gambar 3. Screenshot Output Program Soal 3.....	12
Gambar 4. Screenshot Output Program Soal 4.....	13
Gambar 5. Screenshot Output Program Soal 5.....	14
Gambar 6. Screenshot Output Program Soal 6.....	15
Gambar 7. Screenshot Output Program Soal 7.....	16
Gambar 8. Screenshot Output Program Soal 8.....	17
Gambar 9. Screenshot Output Program Soal 9.....	18
Gambar 10. Screenshot Output Program Soal 10.....	19

DAFTAR TABEL

Tabel 1. Source Code Soal Nomor 1	2
---	---

SOAL 1

Soal

1. Cobalah program berikut, running, simpan program, dan screenshoot hasil running !

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  #include <map>
5  #include <algorithm>
6  #include <stack>
7  #include <queue>
8  #include <set>
9  #include <unordered_set>
10 #include <unordered_map>
11 #include <memory>
12 #include <limits>
13 #include <climits>
14 #include <cmath>
15 #include <cstdio>
16 #include <cstdlib>
17 #include <cstring>
18 #include <ctime>
19 #include <chrono>
20 #include <random>
21 #include <bitset>
22 #include <atomic>
23 #include <mutex>
24 #include <thread>
25 #include <future>
26 #include <promise>
27 #include <condition_variable>
28 #include <shared_mutex>
29 #include <weak_mutex>
30 #include <shared_mutex>
31 #include <weak_mutex>
32 #include <shared_mutex>
33 #include <weak_mutex>
34 #include <shared_mutex>
35 #include <weak_mutex>
36 #include <shared_mutex>
37 #include <weak_mutex>
38 #include <shared_mutex>
39 #include <weak_mutex>
40 #include <shared_mutex>
41 #include <weak_mutex>
42 #include <shared_mutex>
43 #include <weak_mutex>
44 #include <shared_mutex>
45 #include <weak_mutex>
46 #include <shared_mutex>
47 #include <weak_mutex>
48 #include <shared_mutex>
49 #include <weak_mutex>
50 #include <shared_mutex>
51 #include <weak_mutex>
52 #include <shared_mutex>
53 #include <weak_mutex>
54 #include <shared_mutex>
55 #include <weak_mutex>
56 #include <shared_mutex>
57 #include <weak_mutex>
58 #include <shared_mutex>
59 #include <weak_mutex>
60 #include <shared_mutex>
61 #include <weak_mutex>
62 #include <shared_mutex>
63 #include <weak_mutex>
64 #include <shared_mutex>
65 #include <weak_mutex>
66 #include <shared_mutex>
67 #include <weak_mutex>
68 #include <shared_mutex>
69 #include <weak_mutex>
70 #include <shared_mutex>
71 #include <weak_mutex>
72 #include <shared_mutex>
73 #include <weak_mutex>
74 #include <shared_mutex>
75 #include <weak_mutex>
76 #include <shared_mutex>
77 #include <weak_mutex>
78 #include <shared_mutex>
79 #include <weak_mutex>
80 #include <shared_mutex>
81 #include <weak_mutex>
82 #include <shared_mutex>
83 #include <weak_mutex>
84 #include <shared_mutex>
85 #include <weak_mutex>
86 #include <shared_mutex>
87 #include <weak_mutex>
88 #include <shared_mutex>
89 #include <weak_mutex>
90 #include <shared_mutex>
91 #include <weak_mutex>
92 #include <shared_mutex>
93 #include <weak_mutex>
94 #include <shared_mutex>
95 #include <weak_mutex>
96 #include <shared_mutex>
97 #include <weak_mutex>
98 #include <shared_mutex>
99 #include <weak_mutex>
100 #include <shared_mutex>

```

[illegible][illegible][illegible][illegible][illegible][illegible][illegible]

```

255         return count == 0
256
257     if (count == 0) {
258         fail = true;
259     }
260
261     cout << "Data yang akan dimasukkan ke dalam data yang beresultasi\n";
262     while (
263         count < total_kardus) data.push_back("card" + to_string(count));
264     } else {
265         cout << "Tidak terdapat data yang beresultasi\n";
266     }
267 }

```

A. Source Code

Tabel 1. Source Code Soal Nomor 1

```
#include <iostream>
#include <cstdlib>
#include <string>

using namespace std;

struct TNode {
    string data;
    TNode *next;
};

TNode *head, *tail;

int pil;
string dataBaru, dataHapus;

void init();
int isEmpty();

void tambahDepan();
void tambahBelakang();
void hapusDepan();
void hapusBelakang();
void tampilkan();
void reset();
void cariData();
void hapusData();
void sisipkanSebelum();
void sisipkanSetelah();

int main()
{
    init();

    do {
        cout << "Single Linked List Circular
(SLLC)\n";
        cout <<
"=====\n";
        cout << "1. Tambah Depan\n";
        cout << "2. Tambah Belakang\n";
        cout << "3. Hapus Depan\n";
```



```

        cout << "4. Hapus Belakang\n";
        cout << "5. Tampilkan Data\n";
        cout << "6. Hapus Semua Elemen\n";
        cout << "7. Cari Data\n";
        cout << "8. Hapus Data Tertentu\n";
        cout << "9. Sisipkan Sebelum\n";
        cout << "10. Sisipkan Setelah\n";
        cout << "11. Quit\n";
        cout << "Pilihan : ";
        cin >> pil;

        switch (pil) {
        case 1: tambahDepan(); break;
        case 2: tambahBelakang(); break;
        case 3: hapusDepan(); break;
        case 4: hapusBelakang(); break;
        case 5: tampilkan(); break;
        case 6: reset(); break;
        case 7: cariData(); break;
        case 8: hapusData(); break;
        case 9: sisipkanSebelum(); break;
        case 10: sisipkanSetelah(); break;
        default:
            cout << "\nTERIMA KASIH\n";
        }

        cout << "\nTekan ENTER...";
        cin.ignore();
        cin.get();

        system("cls");

    } while (pil != 11);

    return 0;
}

void init() {
    head = NULL;
    tail = NULL;
}

int isEmpty() {
    return (head == NULL);
}

```

```

void tambahDepan() {
    TNode *baru = new TNode;
    cout << "Masukkan data : ";
    cin >> dataBaru;

    baru->data = dataBaru;

    if (isEmpty()) {
        head = tail = baru;
        baru->next = head;
    } else {
        baru->next = head;
        head = baru;
        tail->next = head;
    }

    cout << "Berhasil tambah depan\n";
}

void tambahBelakang() {
    TNode *baru = new TNode;
    cout << "Masukkan data : ";
    cin >> dataBaru;

    baru->data = dataBaru;

    if (isEmpty()) {
        head = tail = baru;
        baru->next = head;
    } else {
        tail->next = baru;
        tail = baru;
        tail->next = head;
    }

    cout << "Berhasil tambah belakang\n";
}

void hapusDepan() {
    if (!isEmpty()) {
        TNode *hapus = head;
        dataHapus = hapus->data;

        if (head != tail) {

```

```

        head = head->next;
        tail->next = head;
    } else {
        init();
    }

    delete hapus;
    cout << "Hapus depan: " << dataHapus <<
endl;
    } else {
        cout << "List kosong\n";
    }
}

void hapusBelakang() {
    if (!isEmpty()) {
        TNode *hapus = tail;
        dataHapus = hapus->data;

        if (head != tail) {
            TNode *bantu = head;
            while (bantu->next != tail)
                bantu = bantu->next;

            tail = bantu;
            tail->next = head;
        } else {
            init();
        }

        delete hapus;
        cout << "Hapus belakang: " << dataHapus
<< endl;
    } else {
        cout << "List kosong\n";
    }
}

void tampilkan() {
    if (!isEmpty()) {
        TNode *bantu = head;
        do {
            cout << bantu->data << " ";
            bantu = bantu->next;
        } while (bantu != head);
    }
}

```

```

        cout << endl;
    } else {
        cout << "List kosong\n";
    }
}

void reset() {
    while (!isEmpty()) {
        hapusDepan();
    }
    cout << "Semua data dihapus\n";
}

void cariData() {
    if (!isEmpty()) {
        string cari;
        cout << "Cari data: ";
        cin >> cari;

        TNode *bantu = head;
        bool ketemu = false;

        do {
            if (bantu->data == cari) {
                ketemu = true;
                break;
            }
            bantu = bantu->next;
        } while (bantu != head);

        if (ketemu) cout << "Data ditemukan\n";
        else cout << "Tidak ditemukan\n";
    } else {
        cout << "List kosong\n";
    }
}

void hapusData() {
    if (!isEmpty()) {
        string cari;
        cout << "Hapus data: ";
        cin >> cari;

        TNode *bantu = head;
        TNode *sebelum = tail;
    }
}

```

```

        bool ditemukan = false;

        do {
            if (bantu->data == cari) {
                ditemukan = true;

                if (bantu == head) {
                    hapusDepan();
                    bantu = head;
                    sebelum = tail;
                } else if (bantu == tail) {
                    hapusBelakang();
                    bantu = head;
                    sebelum = tail;
                } else {
                    sebelum->next = bantu->next;
                    delete bantu;
                    bantu = sebelum->next;
                }
            } else {
                sebelum = bantu;
                bantu = bantu->next;
            }

        } while (bantu != head);

        if (ditemukan) cout << "Data berhasil
dihapus\n";
        else cout << "Data tidak ditemukan\n";
    } else {
        cout << "List kosong\n";
    }
}

void sisipkanSebelum() {
    if (!isEmpty()) {
        string target;
        cout << "Sisip sebelum: ";
        cin >> target;

        TNode *bantu = head;
        TNode *sebelum = tail;

        do {
            if (bantu->data == target) {

```

```

        TNode *baru = new TNode;
        cout << "Data baru: ";
        cin >> dataBaru;

        baru->data = dataBaru;
        baru->next = bantu;
        sebelum->next = baru;

        if (bantu == head)
            head = baru;

        cout << "Berhasil sisip\n";
        return;
    }
    sebelum = bantu;
    bantu = bantu->next;
} while (bantu != head);

    cout << "Data tidak ditemukan\n";
} else {
    cout << "List kosong\n";
}
}

void sisipkanSetelah() {
    if (!isEmpty()) {
        string target;
        cout << "Sisip setelah: ";
        cin >> target;

        TNode *bantu = head;

        do {
            if (bantu->data == target) {
                TNode *baru = new TNode;
                cout << "Data baru: ";
                cin >> dataBaru;

                baru->data = dataBaru;
                baru->next = bantu->next;
                bantu->next = baru;

                if (bantu == tail)
                    tail = baru;
            }
        } while (bantu->next != null);
    }
}

```

```

        cout << "Berhasil sisip\n";
        return;
    }
    bantu = bantu->next;
} while (bantu != head);

cout << "Data tidak ditemukan\n";
} else {
    cout << "List kosong\n";
}
}

```

B. Output Program

```

PS C:\Users\LEGION\Downloads\codingan algo\modul_3> cd 'C:\Users\LEGION\Downloads\codingan algo\modul_3\output'
PS C:\Users\LEGION\Downloads\codingan algo\modul_3\output> & .\code.3.exe
Single Linked List Circular (SLLC)
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Data Tertentu
9. Sisipkan Sebelum
10. Sisipkan Setelah
11. Quit
Pilihan :

```

Gambar1. Screenshot Output Program Soal 1

C. Pembahasan

Program Single Linked List Circular (SLLC) berhasil dijalankan setelah dilakukan proses compile tanpa mengalami error. Program ini dibuat menggunakan bahasa pemrograman C++ dan memanfaatkan struktur data linked list berbentuk circular, yaitu node terakhir (tail) akan menunjuk kembali ke node pertama (head). Saat program dijalankan, akan ditampilkan menu utama yang berisi beberapa pilihan operasi, seperti menambah data di depan, menambah data di belakang, menghapus data di depan, menghapus data di belakang, menampilkan seluruh data, menghapus semua data, mencari data, menghapus data tertentu, serta menyisipkan data sebelum dan sesudah node tertentu.

Pengguna diminta untuk memasukkan pilihan menu sesuai dengan operasi yang diinginkan. Setelah pengguna memilih menu, program akan menjalankan fungsi sesuai dengan pilihan tersebut. Setiap operasi yang dilakukan akan memberikan output berupa informasi keberhasilan atau hasil dari proses tersebut, seperti data berhasil ditambahkan, data berhasil dihapus, atau hasil pencarian data.

Selain itu, program juga menggunakan perulangan (looping) sehingga menu akan terus ditampilkan kembali setelah satu operasi selesai dilakukan. Hal ini memungkinkan pengguna untuk melakukan berbagai operasi secara berulang tanpa harus menjalankan ulang program dari awal.

Program akan berhenti atau keluar ketika pengguna memilih opsi keluar (Quit), yaitu pada pilihan nomor 11. Sebelum kembali ke menu, program akan meminta pengguna menekan tombol ENTER sebagai jeda, sehingga pengguna dapat melihat hasil dari operasi yang telah dilakukan.

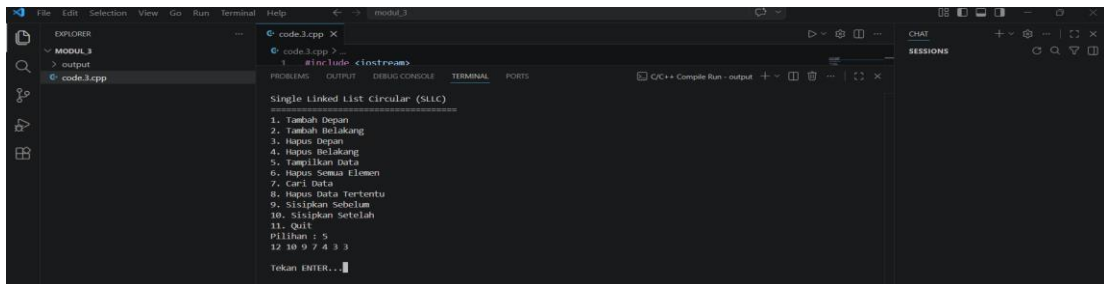
Dengan demikian, program ini sudah berjalan dengan baik dan mampu mengimplementasikan operasi dasar pada struktur data Single Linked List Circular sesuai dengan yang diharapkan.

SOAL 2

Soal

- Lakukan tambah data depan 3, 4, 7, 9, 10, 12 dan kemudian lakukan tampilkan data lalu screenshoot hasilnya !

A. Output Program



```
code.3.cpp x
1 #include <iostream>
Single Linked List Circular (SLLC)
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Data Tertentu
9. Sisipkan Sebelum
10. Sisipkan Setelah
11. Quit
Pilihan : 5
12 10 9 7 4 3
Tekan ENTER...
```

Gambar2. Screenshot Output Program Soal 2

B. Pembahasan

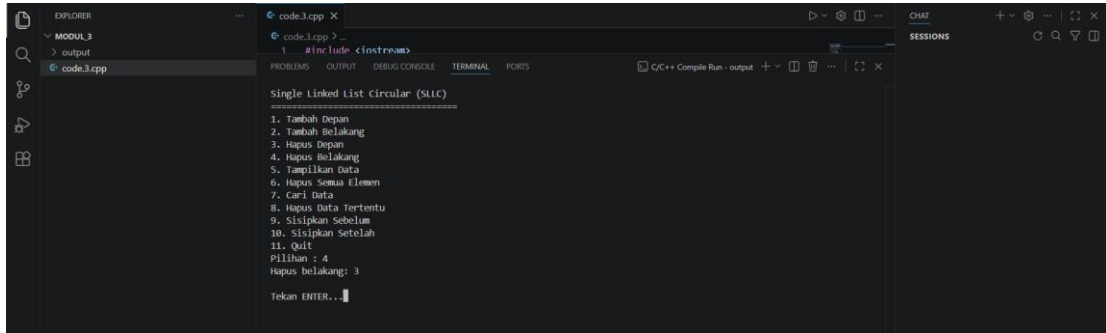
Penambahan data di bagian depan dilakukan melalui menu tambah depan, dimana setiap data yang dimasukkan akan langsung ditempatkan sebagai node pertama atau head. Proses ini menyebabkan data yang sebelumnya berada di depan akan bergeser ke belakang. Ketika data dimasukkan secara berurutan yaitu 3, 4, 7, 9, 10, dan 12, maka setiap data baru selalu berada di posisi paling depan. Akibatnya, urutan data yang ditampilkan menjadi 12 10 9 7 4 3. Hal ini menunjukkan bahwa metode tambah depan menghasilkan urutan data yang terbalik dari urutan input.

SOAL 3

Soal

3. Lakukan tambah data belakang 3, 7, 1, 4, 3 dan kemudian lakukan tampilkan data lalu screenshoot hasilnya !

A. Output Program



```
code3.cpp X
code3.cpp > ...
1 #include <iostream>

Single Linked List Circular (SLLC)
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Data Tertentu
9. Sisipkan Sebelum
10. Sisipkan Setelah
11. Quit
Pilihan : 4
Hapus belakang: 3
Tekan ENTER...
```

Gambar3. Screenshot Output Program Soal 3

B. Pembahasan

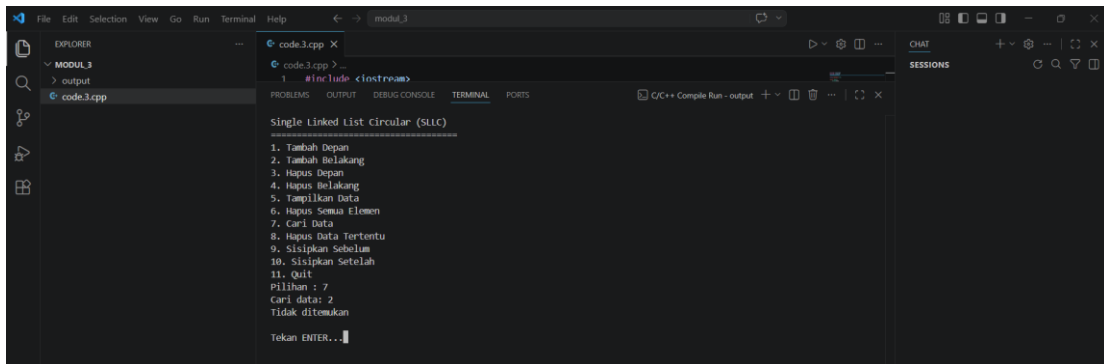
Penambahan data di bagian belakang dilakukan melalui menu tambah belakang, dimana setiap data baru akan ditempatkan di posisi terakhir atau tail. Data yang sudah ada sebelumnya tidak berubah posisinya. Ketika data dimasukkan dengan urutan 3, 7, 1, 4, dan 3, maka hasil yang ditampilkan tetap sama yaitu 3 7 1 4 3. Hal ini terjadi karena setiap data ditambahkan di bagian akhir sehingga urutan tetap mengikuti input awal.

SOAL 4

Soal

4. Apa yang terjadi jika mencari angka 2 pada Single Linked List Circular (SLLC) pada data yang telah ditambahkan/dimasukkan sebelumnya dan screenshoot hasilnya

A. Output Program



```
code3.cpp X
1 //include <iostream>
2
3 Single linked list circular (SLLC)
4
5 1. Tambah Depan
6 2. Tambah Belakang
7 3. Hapus Depan
8 4. Hapus Belakang
9 5. Tampilkan Data
10 6. Hapus Semua Elemen
11 7. Cari Data
12 8. Hapus Data Tertentu
13 9. Sisipkan Sebelum
14 10. Sisipkan Setelah
15 11. Quit
16
17 Pilihan : 7
18 Cari data: 2
19 Tidak ditemukan
20 Tekan ENTER...
```

Gambar4. Screenshot Output Program Soal 4

B. Pembahasan

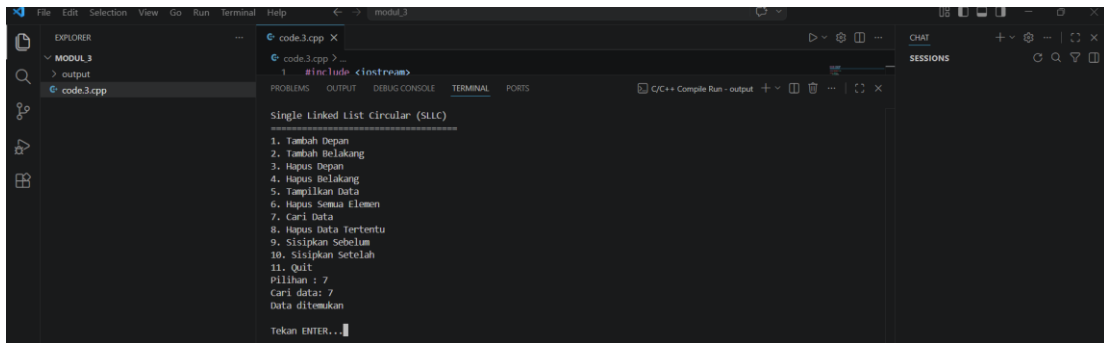
Pada saat dilakukan pencarian terhadap data bernilai 2, program akan menelusuri seluruh node dimulai dari head hingga kembali lagi ke head. Setiap data dibandingkan satu per satu dengan nilai yang dicari. Karena angka 2 tidak terdapat dalam linked list, maka program akan menampilkan hasil bahwa data tidak ditemukan. Hal ini menunjukkan bahwa proses pencarian berjalan dengan baik hingga seluruh data diperiksa.

SOAL 5

Soal

5. Coba cari angka 7 dan screenshoot hasilnya !

A. Output Program



```
code3.cpp X
code3.cpp > ...
#include <iostream>

Single Linked List Circular (SLLC)

1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Data Tertentu
9. Sisipkan Sebelum
10. Sisipkan Setelah
11. QUIT
Pilihan : 7
Cari data: 7
Data ditemukan

Tekan ENTER...
```

Gambar5. Screenshot Output Program Soal 5

B. Pembahasan

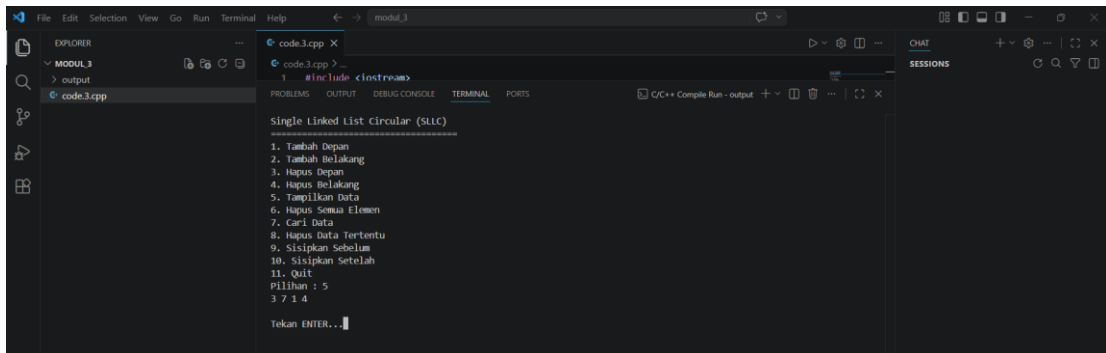
Ketika dilakukan pencarian terhadap data bernilai 7, program akan menelusuri node satu per satu hingga menemukan data yang sesuai. Karena angka 7 terdapat dalam linked list, maka program akan menghentikan pencarian dan menampilkan bahwa data ditemukan. Hal ini menunjukkan bahwa proses pencarian berhasil mendeteksi data yang ada dalam struktur linked list.

SOAL 6

Soal

6. Lakukan hapus belakang dan kemudian lakukan tampilkan data lalu screenshoot hasilnya !

A. Output Program



```
code.3.cpp X
1 #include <iostream>

Single Linked List Circular (SLLC)
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua elemen
7. Cari Data
8. Hapus Data Tertentu
9. Sisipkan Sebelum
10. Sisipkan Setelah
11. Quit
Pilihan : 5
3 7 1 4
Tekan ENTER...
```

Gambar6. Screenshot Output Program Soal 6

B. Pembahasan

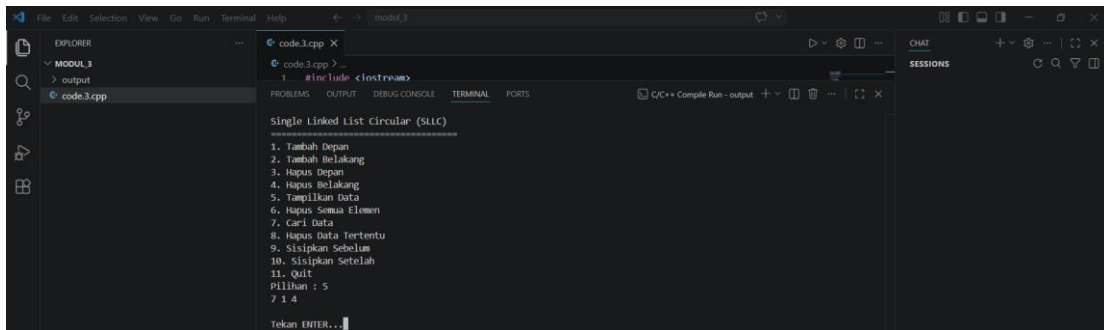
Pada saat dilakukan penghapusan data di bagian belakang, node terakhir atau tail akan dihapus dari linked list. Setelah itu, node sebelumnya akan menjadi tail yang baru. Selain itu, hubungan antara tail dan head tetap dijaga agar struktur tetap berbentuk circular. Setelah penghapusan, jumlah data akan berkurang satu dan data terakhir sebelumnya tidak lagi ditampilkan.

SOAL 7

Soal

7. Lakukan hapus setiap angka 3 dan kemudian lakukan tampilkan data lalu screenshoot hasilnya !

A. Output Program



```
code.3.cpp x
1 #include <iostream>

Single Linked List Circular (SLLC)
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua elemen
7. Cari Data
8. Hapus Data Tertentu
9. Sisipkan Sebelum
10. Sisipkan Setelah
11. Quit
Pilihan : 5
7 1 4
Tekan ENTER...
```

Gambar7. Screenshot Output Program Soal 7

B. Pembahasan

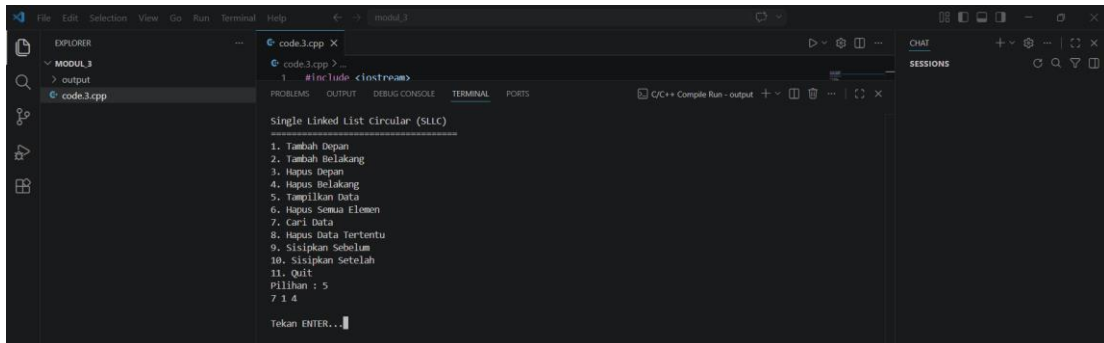
Pada proses penghapusan data tertentu, program akan menelusuri seluruh node dan menghapus semua node yang memiliki nilai tertentu, yaitu angka 3. Proses ini dilakukan dengan hati-hati karena node yang dihapus bisa berada di posisi depan, tengah, maupun belakang. Setelah semua data bernilai 3 dihapus, maka data yang tersisa hanya yang tidak memiliki nilai tersebut. Hal ini menunjukkan bahwa fungsi penghapusan data tertentu dapat bekerja dengan baik untuk lebih dari satu data yang sama.

SOAL 8

Soal

8. Tampilkan data lalu jelaskan yang mana head dan yang mana tail.

A. Output Program



```
code3.cpp x
code3.cpp > ...
1 #include <iostream>

Single Linked List Circular (SLLC)
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Data Tertentu
9. Sisipkan Sebelum
10. Sisipkan Setelah
11. Quit
Pilihan : 5
7 1 4
Tekan ENTER...
```

Gambar8. Screenshot Output Program Soal 8

B. Pembahasan

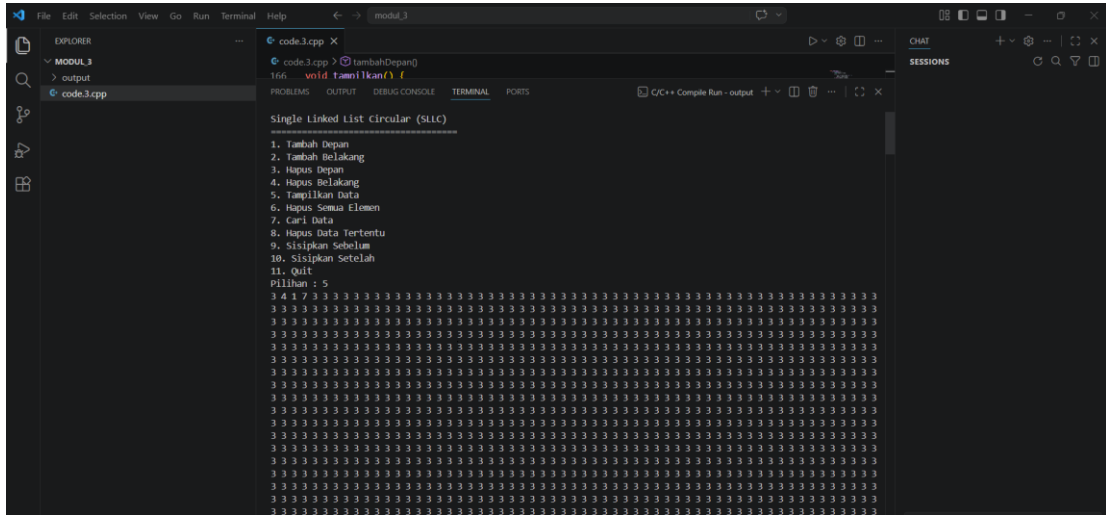
Pada saat data ditampilkan, node pertama yang muncul merupakan head, yaitu titik awal dari linked list. Sedangkan node terakhir merupakan tail, yaitu node yang berada di posisi paling akhir. Sebagai contoh, jika data yang ditampilkan adalah 7 1 4, maka 7 merupakan head dan 4 merupakan tail. Head digunakan sebagai titik awal untuk mengakses seluruh data dalam linked list. Pada struktur Single Linked List Circular, tail memiliki perbedaan dibandingkan linked list biasa, yaitu tail tidak menunjuk ke NULL, melainkan menunjuk kembali ke head. Hal ini menyebabkan struktur data membentuk lingkaran, sehingga proses penelusuran dapat dilakukan secara terus-menerus.

SOAL 9

Soal

9. Jika baris ke 103 dan 104 dihapus maka apa yang akan terjadi pada saat memasukkan data, dan jelaskan mengapa?

A. Output Program



Gambar9. Screenshot Output Program Soal 9

B. Pembahasan

Jika baris yang berfungsi menghubungkan node terakhir ke node pertama dihapus, maka struktur Single Linked List Circular tidak lagi terbentuk. Node terakhir tidak akan menunjuk kembali ke head, sehingga linked list berubah menjadi tidak melingkar.

Akibatnya, saat memasukkan data, hubungan antar node tidak membentuk siklus seperti seharusnya. Hal ini dapat menyebabkan proses penelusuran data menjadi tidak berjalan dengan benar, karena program masih menggunakan logika circular yang mengharuskan node terakhir kembali ke node pertama.

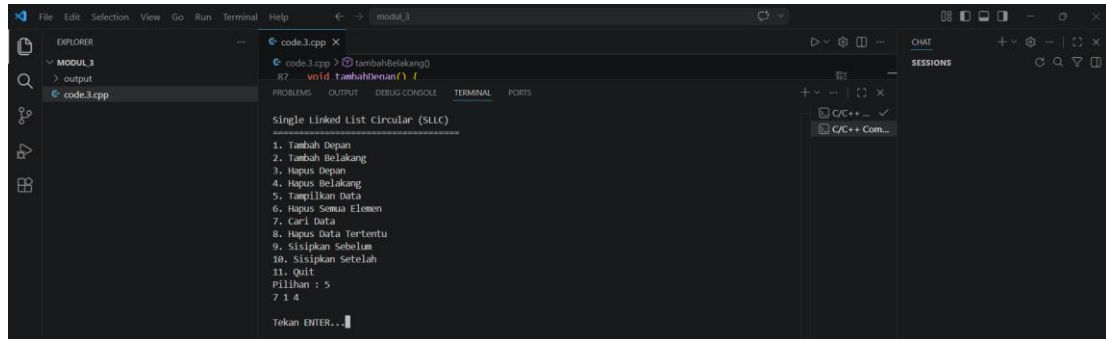
Selain itu, kondisi ini juga dapat menyebabkan kesalahan pada saat menampilkan data atau melakukan operasi lainnya, karena alur pointer tidak sesuai dengan konsep circular linked list. Dengan demikian, baris tersebut sangat penting untuk menjaga struktur linked list tetap berbentuk circular.

SOAL 10

Soal

10. Jelaskan apa itu variabel head dan tail pada sllc!

A. Output Program



```
code3.cpp X
code3.cpp > (tambahbelakang)
R7 void tambahdepan() {
Single Linked List Circular (SLLC)
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Data Tertentu
9. Sisipkan Sebelum
10. Sisipkan Setelah
11. Quit
Pilihan : 5
7 1 4
Tekan ENTER...
```

Gambar10. Screenshot Output Program Soal 10

B. Pembahasan

Variabel head dan tail merupakan dua pointer utama dalam struktur Single Linked List Circular yang memiliki peran penting dalam mengelola data.

Head digunakan untuk menunjuk node pertama dalam linked list dan menjadi titik awal dalam proses penelusuran data. Sedangkan tail menunjuk node terakhir yang berfungsi untuk menghubungkan kembali ke head sehingga membentuk struktur melingkar. Sebagai contoh, jika data yang ditampilkan adalah 7 1 4, maka 7 merupakan head dan 4 merupakan tail. Tail akan menunjuk kembali ke head sehingga data membentuk lingkaran.

Dengan adanya head dan tail, proses penambahan, penghapusan, dan penelusuran data dapat dilakukan dengan lebih mudah dan terstruktur.

LINK GITHUB

https://github.com/Monkey0909-ai/laprak_algo